MARUTI FABRISCAN

Frontend Product, UI/UX & Interaction Specification

Project: Maruti FabriScan — Edge Automated Optical Inspection
Document Type: Frontend Product & UI/UX Specification
Reference: MSIL-FS-SRS-2026-V1.0
Frontend Status: Proposed Design Baseline
Primary Surfaces: Desktop Web / Industrial Panel PC / Tablet / Smartphone
Design Principle: Zero additional specialized AOI hardware
Primary Objective: Real-time industrial visual inspection with immediate, understandable and safe operator feedback

---

1. Document Purpose

This document defines the complete frontend experience for Maruti FabriScan.

It covers:

- frontend technology stack
- application architecture
- information architecture
- screen structure
- navigation
- live inspection experience
- camera experience
- AI visualization
- defect presentation
- production metrics
- alerts
- rejection states
- mobile experience
- desktop experience
- accessibility
- iconography
- typography
- colors
- spacing
- components
- buttons
- tables
- charts
- forms

- modals
- notifications
- loading states
- empty states
- error states
- offline behavior
- role-based UI
- interaction behavior
- responsive behavior
- frontend/backend communication
- WebSocket behavior
- browser/WASM considerations
- implementation structure
- QA requirements

The frontend should not feel like a generic SaaS dashboard.

It is an **industrial operator interface**.

The primary question behind every screen is:

> **Can an operator understand the current production state and take the correct action within seconds, without ambiguity?**

---

2. Product Context

FabriScan is an edge-based Automated Optical Inspection system intended for automotive manufacturing environments.

The SRS defines the system around:

1. camera acquisition
2. frame normalization
3. edge AI inference
4. defect segmentation
5. defect measurement
6. severity classification
7. rejection actuation
8. production metrics
9. supplier traceability
10. audit/report generation

The frontend is therefore not simply a visualization layer.

It is the **human control and interpretation layer of the inspection system**.

The SRS requires a split camera experience, dark industrial UI, high contrast, real-time segmentation rendering, operator controls and production metrics.

---

3. Frontend Design Principles

3.1 Safety Before Aesthetics

A critical rejection must be visually unmistakable.

The system should never depend on:

- color alone
- tiny text
- subtle animation
- hidden notifications
- ambiguous icons

for communicating a production-critical state.

3.2 Operator First

The Line QC Inspector is the primary user of the live screen.

The UI must therefore prioritize:

1. current line state
2. current camera state
3. current inspected part
4. current defect
5. rejection state
6. action required from operator

Historical analytics should not visually compete with the live inspection state.

3.3 Glanceability

An operator should be able to answer these questions immediately:

- Is the line running?
- Is the camera working?
- Is AI inference running?
- Is the current panel acceptable?

- Is a defect present?
- What defect is it?
- How severe is it?
- Was rejection triggered?
- Does the operator need to intervene?

3.4 Industrial Visual Language

The visual language should communicate:

- precision
- engineering
- reliability
- instrumentation
- manufacturing
- machine vision

Avoid:

- excessive gradients
- decorative illustrations
- excessive rounded cards
- consumer-app styling
- oversized marketing graphics
- unnecessary animation

---

4. Recommended Frontend Technology Stack

The following is the **proposed frontend stack**. The SRS specifies the browser/WASM environment but does not mandate a particular UI framework, so these choices are implementation recommendations rather than existing SRS requirements.

4.1 Core

React

Use React for the application UI.

Reason:

- component-based architecture
- mature ecosystem
- suitable for complex dashboards

- strong TypeScript integration
- efficient state-driven rendering
- good separation between inspection engine and UI

4.2 Language

TypeScript

All frontend application code should use TypeScript.

No new production feature should be written in plain JavaScript.

TypeScript should define:

- camera states
- defect objects
- severity enums
- inspection events
- batch objects
- WebSocket messages
- API responses
- user roles
- system states

---

5. Build Tool

Vite

Recommended build system:

Vite + React + TypeScript

Requirements:

- fast development server
- production bundling
- code splitting
- Web Worker support
- WASM asset handling
- environment-specific configuration

---

6. Styling System

Tailwind CSS

Recommended for utility-based layout and responsive styling.

However, Tailwind should be combined with a **FabriScan design-token layer**.

Do not allow developers to arbitrarily choose:

```
gray-700
gray-800
blue-500
rounded-xl
text-sm
```

throughout the application.

Instead, define semantic tokens.

Example:

```
--fs-surface-primary
--fs-surface-secondary
--fs-border-subtle

--fs-text-primary
--fs-text-secondary
--fs-text-muted

--fs-status-running
--fs-status-warning
--fs-status-critical
--fs-status-success

--fs-action-primary
--fs-action-danger
```

---

7. Component Library

Recommended:

shadcn/ui + Radix primitives

Use the primitives for:

- Dialog
- Dropdown
- Tooltip
- Popover
- Tabs
- Select
- Toast
- Checkbox
- Switch
- Slider
- Scroll Area

FabriScan-specific components should be built on top of them.

Do not expose generic component styling directly to the operator.

---

8. Icon Library

Recommended: Lucide Icons

Use a single consistent icon family.

Recommended icons:

Function	Icon
Dashboard	`LayoutDashboard`
Live Inspection	`ScanLine`
Camera	`Camera`
Camera disconnected	`CameraOff`
AI	`BrainCircuit`
Settings	`Settings`
Calibration	`Crosshair`
Defect	`TriangleAlert`
Critical defect	`OctagonAlert`

Function	Icon
Success	`CircleCheck`
Warning	`TriangleAlert`
Line running	`PlayCircle`
Line stopped	`CircleStop`
Pause	`PauseCircle`
Search	`Search`
Filter	`ListFilter`
Download	`Download`
Export	`FileDown`
PDF	`FileText`
History	`History`
Analytics	`ChartNoAxesCombined`
Supplier	`Building2`
Users	`Users`
Security	`ShieldCheck`
Network	`Network`
WebSocket	`RadioTower`
Refresh	`RefreshCw`
Reconnect	`PlugZap`
Manual override	`Hand`
Eye/view	`Eye`
Hide	`EyeOff`
Fullscreen	`Maximize2`
Close	`X`
Menu	`Menu`
More actions	`MoreVertical`
Information	`Info`
Help	`CircleHelp`

Icon rules:

- standard icon size: 20px
- compact icon: 16px
- prominent status icon: 24–32px
- critical alert icon: 32–40px
- never use emoji as UI icons
- never mix icon libraries

---

9. Typography

Recommended primary font:

Inter

Use Inter for:

- navigation
- metrics
- labels
- buttons
- tables
- status messages
- forms

Optional secondary typeface:

JetBrains Mono

Use only for technical values:

- FPS
- latency
- timestamps
- confidence
- IDs
- camera stream diagnostics
- WebSocket status
- system telemetry

Example:

```
RUNNING
31 FPS
LATENCY 64.8 ms
```

---

10. Color System

The interface should use a dark industrial base.

Base

```
Background:
#09090B

Primary Surface:
#111113

Secondary Surface:
#18181B

Border:
#27272A

Primary Text:
#F4F4F5

Secondary Text:
#A1A1AA

Muted Text:
#71717A
```

Semantic Colors

Success

```
#16A34A
```

Meaning:

- system healthy
- inspection passed

- camera healthy
- batch within threshold

Warning

```
#F59E0B
```

Meaning:

- degraded FPS
- elevated defect rate
- attention required
- calibration warning

Critical

```
#DC2626
```

Meaning:

- critical defect
- rejection
- line interrupt
- camera failure when fail-safe applies

Information

```
#2563EB
```

Meaning:

- informational
- active selection
- system information

Do not use red for ordinary navigation or decorative emphasis.

---

11. Spacing System

Use a 4px base spacing scale.

```
4px
8px
12px
16px
20px
24px
32px
40px
48px
64px
```

Default UI padding:

- compact: 8px
- standard: 16px
- section: 24px
- major layout: 32px

---

12. Border Radius

Industrial interfaces should avoid excessive rounding.

Recommended:

```
Buttons: 6px
Inputs: 6px
Panels: 8px
Dialogs: 10px
Status indicators: 999px
```

Do not use highly rounded “mobile SaaS” cards throughout the system.

---

13. Application Information Architecture

The frontend should contain these primary areas:

```
FabriScan
│
├── Live Inspection
│
```

```
├── Production
│   ├── Current Batch
│   ├── Batch History
│   └── Quality Metrics
│
├── Defect History
│
├── Reports
│   ├── Inspection Reports
│   └── Non-Conformance Reports
│
├── Calibration
│
├── Camera & Line Configuration
│
├── System Health
│
└── Administration
```

The Line QC Inspector should see fewer administrative options than the System Administrator.

---

14. Global Application Shell

14.1 Top Bar

The top bar should contain:

Left

FabriScan logo/wordmark.

Immediately beside it:

```
PRESS SHOP
LINE 04
```

Example:

```
FABRISCAN
PRESS SHOP / LINE 04
```

Center

Current system state.

Example:

```
● RUNNING
31 FPS
64.8 ms
```

Right

- notification icon
- connection status
- user avatar
- user role
- settings

---

15. Sidebar Navigation

Desktop sidebar width:

240px

Collapsed width:

64px

Navigation item:

```
[icon] Live Inspection
```

Selected item:

- subtle surface highlight
- left 3px active indicator
- white text
- icon emphasized

Do not use excessive glowing effects.

---

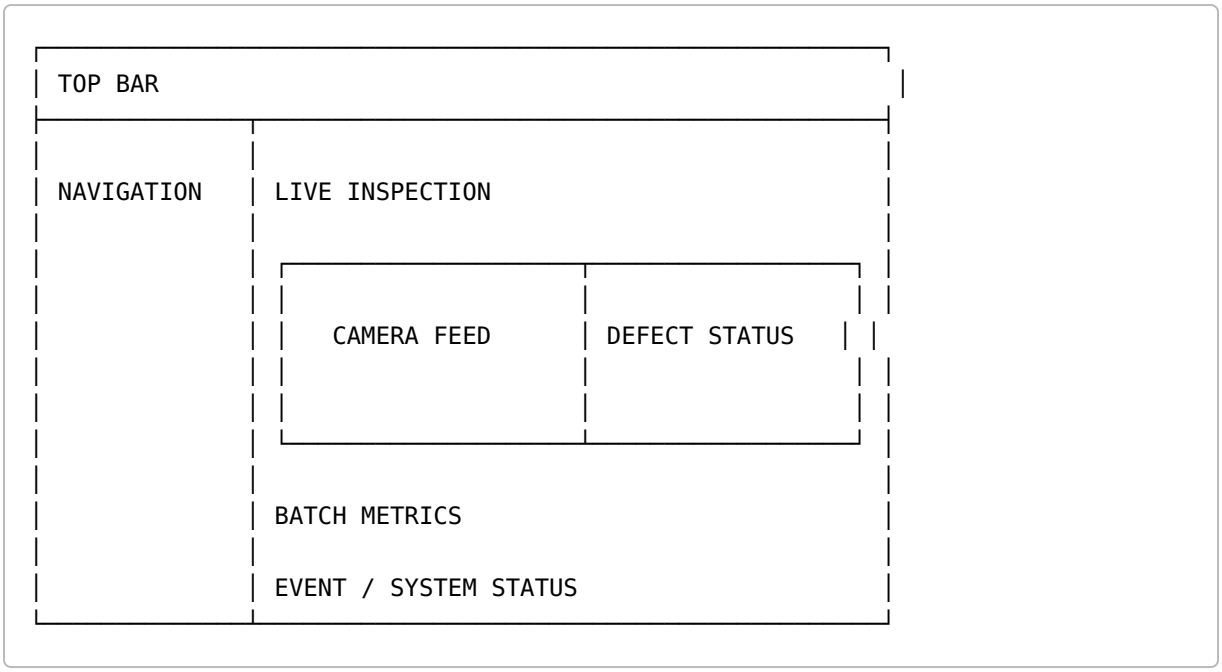
16. Screen 1 — Live Inspection

This is the most important screen.

The majority of frontend design effort should go here.

16.1 Layout

Desktop:



---

17. Camera Viewport

The camera viewport is the visual center of the application.

It should support:

- raw video
- segmentation overlay
- bounding contours
- defect labels
- confidence
- measurement annotation
- tracking indicator

- frame status

---

18. Camera Header

Above the camera:

```
CAMERA 01
● LIVE
1920 × 1080
31 FPS
```

Additional actions:

- fullscreen
- snapshot
- overlay toggle
- camera settings

Icons:

- `Maximize2`
- `Camera`
- `Layers3`
- `Settings`

---

19. Segmentation Overlay

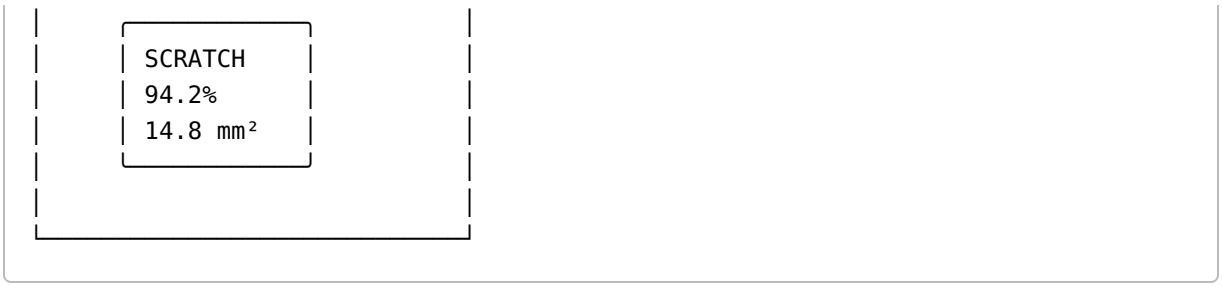
Detected defects should be rendered directly on the camera image.

Each defect should have:

1. mask
2. contour
3. class label
4. confidence
5. severity indicator

Example:

```
┌──────────────────────────────┐
│                              │
```



The mask should be translucent enough that the underlying metal surface remains visible.

---

20. Defect Label Design

Do not place enormous labels over the defect.

Recommended structure:

```
SCRATCH
94.2%
14.8 mm²
CRITICAL
```

Class:

14px semibold

Confidence:

12px mono

Area:

12px mono

Severity:

12px uppercase

---

21. Defect Colors

Defect visualization should use category/severity carefully.

The primary semantic color should represent severity.

Minor

Amber/neutral warning treatment.

Major

Orange treatment.

Critical

Red treatment.

The UI should not rely solely on mask color.

Every defect must also have:

- text class
- severity
- icon/status indicator

---

22. Active Anomaly Panel

Right-side panel.

Header:

```
ACTIVE ANOMALY
```

If nothing is detected:

```
NO ACTIVE DEFECT

Inspection clear
```

Use:

`CircleCheck`

If defect exists:

```
ROLLED-IN SCALE

Confidence
94.2%

Surface Area
14.8 mm²

Severity
CRITICAL

ACTION
REJECTION ACTUATED
```

---

23. Critical Rejection State

When a critical defect occurs, the entire application should transition into a visually stronger state.

Top status:

```
● CRITICAL DEFECT
```

Camera:

- defect highlighted
- subtle pulse around defect
- no distracting screen-wide animation

Right panel:

```
CRITICAL DEFECT

SCRATCH

14.82 mm²
94.12% confidence

REJECTION ACTUATED
```

Primary action:

```
REVIEW EVENT
```

Secondary:

```
MANUAL OVERRIDE
```

Manual override must require deliberate interaction.

---

24. Global Alert Banner

For critical events:

```
┌──────────────────────────────────────────┐
│ ⛔ CRITICAL DEFECT DETECTED              │
│ Rejection signal dispatched • Review inspection │
└──────────────────────────────────────────┘
```

The SRS explicitly requires a high-contrast warning banner and audible alert for critical defects.

---

25. Production Metrics

The live screen should expose only the metrics necessary for immediate awareness.

Primary metrics:

Inspected

```
4,812
```

Defective

```
14
```

FTR

```
99.71%
```

PPM

```
2,909
```

These correspond to the SRS dashboard concept.

---

26. Metric Card Rules

Metrics should not become giant decorative cards.

Use a compact instrumentation layout:

```
INSPECTED
4,812

DEFECTIVE
14

FTR
99.71%

PPM
2,909
```

Each metric should contain:

- label
- value
- optional delta
- optional threshold state

---

27. Line Status

Possible states:

RUNNING

```
● RUNNING
```

WARNING

```
▲ DEGRADED
```

LINE INTERRUPT

```
■ LINE INTERRUPTED
```

STOPPED

```
■ STOPPED
```

INITIALIZING

```
◌ INITIALIZING
```

OFFLINE

```
○ OFFLINE
```

---

28. Camera Failure State

If the camera disconnects:

```
CAMERA CONNECTION LOST

Inspection has been interrupted.

Reconnect camera
```

The interface must make clear that the inspection state is no longer trustworthy.

This corresponds to the SRS fail-safe requirement.

---

29. Low-FPS State

If FPS falls below the defined threshold:

```
INSPECTION QUALITY DEGRADED

Camera stream: 12 FPS
Required minimum: 15 FPS

LINE INTERRUPT WARNING
```

Do not merely show a tiny warning badge.

---

30. AI Engine Status

System health should expose:

```
AI ENGINE
● READY

MODEL
YOLOv8-Seg INT8

INFERENCE
32.5 ms

WASM
SIMD ENABLED

WEBGPU
ENABLED
```

This should primarily be available in diagnostics, not constantly displayed to operators.

---

31. Mobile Frontend

Mobile is important because FabriScan is intended to work with commodity smartphone cameras where appropriate.

The mobile interface should **not** simply shrink the desktop UI.

It should be restructured.

---

32. Mobile Navigation

Use bottom navigation.

Recommended:

```
Live
History
Reports
System
```

Primary action remains:

Live

Do not expose ten navigation items on mobile.

---

33. Mobile Live Inspection

Layout:

```
┌──────────────────────────────┐
│ FABRISCAN      ● RUNNING     │
├──────────────────────────────┤
│                              │
│                              │
│       CAMERA VIEW            │
│                              │
│      DEFECT OVERLAY          │
│                              │
│                              │
├──────────────────────────────┤
│ CRITICAL                     │
│ Scratch • 14.8 mm²           │
│ Confidence 94.2%             │
├──────────────────────────────┤
│ INSPECTED  4812              │
│ FTR 99.71%   PPM 2909        │
├──────────────────────────────┤
```

```
| REVIEW EVENT               |
└────────────────────────────┘
```

---

34. Smartphone Camera Mode

Mobile camera mode should provide:

- camera permission prompt
- rear camera preference
- camera selection when supported
- flashlight control where supported
- focus guidance where supported
- orientation lock
- exposure guidance where supported
- capture/stream status

The interface should clearly state:

```
CAMERA READY
Position the panel inside the inspection zone.
```

---

35. Mobile Capture Guidance

A subtle framing guide can be shown:

```
┌──────────────────────┐
│                      │
│   INSPECTION ZONE    │
│                      │
└──────────────────────┘
```

Avoid heavy AR-style graphics.

---

36. Tablet Experience

Tablet should be treated as a hybrid.

Recommended:

- left navigation
- large camera view
- right-side anomaly panel
- bottom metrics strip

Tablet is particularly useful for:

- QA supervisors
- portable inspections
- calibration
- production review

---

37. Desktop Experience

Desktop/panel PC is the primary continuous-production surface.

Target:

1440 × 1024

Minimum supported responsive layout should accommodate smaller industrial displays.

---

38. Screen 2 — Production Dashboard

Purpose:

Provide supervisors with a production-level overview.

Sections:

Current line state

Current batch

Inspection count

Defect count

FTR

PPM

Defect distribution

Recent critical events

Line performance

---

39. Defect Distribution

Chart:

Horizontal bar chart.

Example:



Avoid pie charts for operational defect analysis.

---

40. Defect History

The defect history table should contain:

- timestamp
- panel ID
- batch ID

- defect class
- confidence
- area
- severity
- rejection status
- review status

Example:

Time	Panel	Defect	Area	Severity	Action
22:15:30	P-4812	Scratch	14.8 mm²	Critical	Rejected
22:14:52	P-4811	Inclusion	2.1 mm²	Major	Rejected

---

41. Defect Detail Drawer

Clicking a defect should open a right-side drawer.

Contents:

```
DEFECT EVENT

Panel ID
PANEL-4812

Batch
BATCH-2026-0927

Class
SCRATCHES

Confidence
94.12%

Area
14.82 mm²

Severity
CRITICAL

Timestamp
22:15:30.124
```

```
Action
REJECTION ACTUATED
```

Below:

- original frame
- segmentation mask
- cropped defect
- event timeline

---

42. Event Playback

The SRS specifies a five-second circular frame buffer.

The UI should expose this as:

```
EVENT PLAYBACK

←──────────●──────→

-5s                    EVENT

[Play]
[Pause]
[Frame -]
[Frame +]
```

Frame stepping should allow the inspector to understand why the system rejected the part.

---

43. Manual Override

Manual override is a safety-sensitive operation.

Do not use:

```
Override
```

as a simple immediate button.

Use:

```
MANUAL OVERRIDE
```

then confirmation:

```
Override rejection?

The system detected:
SCRATCH — CRITICAL

This action will be recorded in the audit trail.

[Cancel]
[Confirm Override]
```

If the role does not have permission:

```
Supervisor authorization required.
```

---

44. Calibration Screen

Calibration should be a dedicated workflow.

Primary information:

```
CAMERA
Camera 01

CURRENT CALIBRATION
K = 12.42 px/mm

REFERENCE OBJECT
100 mm

MEASURED
1242 px
```

Actions:

```
START CALIBRATION
SAVE CALIBRATION
RESET
```

Changing calibration must require appropriate authorization.

---

45. Camera Configuration

Configuration screen:

```
CAMERA 01

Connection
USB / RTSP

Resolution
1280 × 720

Frame Rate
30 FPS

Exposure
AUTO

Status
CONNECTED

Latency
8.2 ms
```

Advanced options should be hidden under:

```
Advanced Camera Settings
```

---

46. Model Configuration

System Administrator view:

```
MODEL

YOLOv8-Seg
INT8

Version
FS-SEG-1.0.4

Runtime
ONNX Runtime Web

Execution
WASM SIMD

Model Status
VALIDATED
```

Model replacement should require confirmation.

---

47. Reports

Reports section:

```
REPORTS

Inspection Reports
Non-Conformance Reports
Batch Reports
Supplier Reports
```

Each report should support:

- date range
- batch
- supplier
- line
- defect class
- severity
- export

---

48. Non-Conformance Report

Report preview:

```
NON-CONFORMANCE REPORT

Supplier
TATA STEEL

Batch
BATCH-STEEL-TATA-9941A

Defect Rate
74 PPM

Threshold
50 PPM

Status
NON-CONFORMING
```

The PDF generation action should be:

```
GENERATE NCR PDF
```

not simply “Download”.

---

49. User Roles

The frontend must respect four roles from the SRS:

Line QC Inspector

Access:

- live inspection
- event review
- false-reject review
- operational controls

QA Supervisor

Additional access:

- calibration
- thresholds
- analytics
- batch reports
- supervisor overrides

Supplier QA

Access:

- supplier portal
- defect evidence
- batch reports
- NCR records

Read-only by default.

System Administrator

Access:

- cameras
- models
- system configuration
- backups
- network
- diagnostics

---

50. Button System

Three primary categories.

Primary

Example:

```
SAVE CALIBRATION
```

Solid action color.

Secondary

Example:

```
VIEW EVENT
```

Subtle border/surface.

Destructive

Example:

```
STOP LINE
```

Red.

Destructive buttons must never be visually indistinguishable from ordinary actions.

---

51. Button Dimensions

Desktop:

```
Height: 40px
Padding: 12–16px
```

Compact:

```
Height: 32px
```

Mobile:

```
Minimum height: 44px
```

Critical operator actions:

```
Minimum height: 48px
```

---

52. Tooltips

Every icon-only button must have a tooltip.

Example:

```
[⛶]
```

Tooltip:

```
Enter fullscreen
```

Never make an operator guess what an icon means.

---

53. Tables

Tables should support:

- sticky header
- sorting
- filtering
- pagination
- row selection
- detail drawer

Critical status should appear as:

```
● CRITICAL
```

not as color alone.

---

54. Notifications

Use two notification classes.

Informational Toast

Short-lived.

Example:

```
Calibration saved successfully.
```

Critical Alert

Persistent until acknowledged.

Example:

```
CRITICAL DEFECT DETECTED
Rejection actuated.
```

Never hide a critical industrial event after a few seconds.

---

55. Loading States

Avoid blank screens.

Use:

```
INITIALIZING CAMERA...
LOADING MODEL...
CONNECTING TO LINE...
SYNCING BATCH...
```

For known loading progress:

```
Loading model
████████████░░ 86%
```

---

56. Empty States

Example:

```
NO DEFECT EVENTS

No defects have been recorded for the selected period.
```

Do not show generic:

```
Nothing here.
```

---

57. Error States

Errors should explain:

1. what happened
2. impact
3. what the operator can do

Example:

```
CAMERA CONNECTION LOST

Inspection is currently unavailable.

Check camera power and network connection.

[RECONNECT CAMERA]
```

---

58. Offline Mode

Because the system must support air-gapped operation, the frontend must not depend on public Internet services.

Do not require:

- Google Fonts at runtime
- cloud APIs
- external analytics
- CDN-hosted critical assets
- external authentication services

All production-critical assets should be locally deployable.

---

59. WebSocket State

The frontend should maintain an explicit connection state:

```
CONNECTING
CONNECTED
DEGRADED
DISCONNECTED
RECONNECTING
```

UI example:

```
● ACTUATOR LINK
CONNECTED
```

When disconnected:

```
⚠ ACTUATOR LINK
DISCONNECTED
```

---

60. REST API Integration

Frontend services should encapsulate API communication.

Example structure:

```
services/
  inspectionService.ts
  batchService.ts
  reportService.ts
  cameraService.ts
  calibrationService.ts
  authService.ts
```

Components should not directly construct API requests.

---

61. WebSocket Event Architecture

Create a central event service.

Example events:

```
DEFECT_DETECTED
REJECTION_ACTUATED
LINE_STATUS_CHANGED
CAMERA_STATUS_CHANGED
FPS_CHANGED
BATCH_UPDATED
SYSTEM_WARNING
```

The UI subscribes to normalized events.

---

62. Frontend State Management

Recommended:

Zustand

Use for application state such as:

```
currentLine
currentBatch
cameraState
inspectionState
activeDefect
systemHealth
userRole
connectionState
```

Do not place every piece of temporary UI state into global state.

Local component state should remain local.

---

63. Web Worker Architecture

Inference-related processing should not block the main UI thread.

Proposed:

```
Main Thread
     │
     ├── UI
     ├── Camera presentation
     └── Interaction
            │
            ▼
       Web Worker
            │
            ├── preprocessing
            ├── WASM inference
            └── postprocessing
```

This is particularly important because the SRS requires real-time rendering and ≤35 ms inference.

---

64. Rendering Architecture

Separate:

Video layer

Raw camera stream.

Overlay layer

Canvas/WebGL.

UI layer

HTML/React.

Conceptually:

```
Video
 ↓
Canvas Overlay
 ↓
HTML UI
```

This prevents React DOM rendering from becoming the bottleneck for every video frame.

65. Animation Rules

Animation should communicate state.

Allowed:

- subtle pulse for live state
- controlled critical alert animation
- progress transitions
- drawer transitions
- hover/focus transitions

Avoid:

- bouncing cards
- continuous decorative animation
- excessive gradients
- spinning elements without meaning

---

66. Accessibility

Minimum requirements:

- keyboard navigation
- visible focus states
- semantic buttons
- accessible labels
- sufficient contrast
- no color-only status
- reduced-motion support
- screen-reader labels for controls

Target contrast should align with the SRS requirement of high-contrast presentation.

---

67. Responsive Breakpoints

Recommended:

```
Mobile
< 640px
```

```
Tablet
640–1023px

Desktop
1024–1439px

Industrial Large
≥ 1440px
```

The exact breakpoints may be adjusted after testing with the target hardware.

---

68. Fullscreen Mode

Live inspection should support fullscreen.

Fullscreen removes:

- unnecessary navigation
- secondary controls
- irrelevant application chrome

and maximizes:

- camera feed
- defect overlay
- active anomaly
- line state

This is the preferred mode for a dedicated shop-floor display.

---

69. Fullscreen Operator Layout

```
┌──────────────────────────────────────────────────────┐
│ LINE 04        ● RUNNING       31 FPS     65 ms    │
├──────────────────────────────────────────────────────┤
│                                                      │
│                                                      │
│                   CAMERA VIEW                        │
│                                                      │
│                [DEFECT MASK]                         │
│                                                      │
```

```
│                                                          │
├──────────────────────────────────────────────────────────┤
│ ACTIVE: SCRATCH      14.8 mm²      CRITICAL              │
└──────────────────────────────────────────────────────────┘
```

---

70. Design Tokens

The project should have one centralized token file.

Example:

```
tokens/
  colors.ts
  spacing.ts
  typography.ts
  radii.ts
  shadows.ts
  motion.ts
```

No random values throughout the codebase.

---

71. Frontend Folder Structure

Recommended:

```
src/
│
├── app/
│   ├── router/
│   ├── providers/
│   └── layout/
│
├── pages/
│   ├── LiveInspection/
│   ├── Production/
│   ├── DefectHistory/
│   ├── Reports/
│   ├── Calibration/
│   ├── CameraConfig/
│   ├── SystemHealth/
```

```
│   └── Administration/
│
├── components/
│   ├── camera/
│   ├── inspection/
│   ├── defects/
│   ├── metrics/
│   ├── alerts/
│   ├── tables/
│   ├── forms/
│   └── navigation/
│
├── services/
│   ├── api/
│   ├── websocket/
│   └── camera/
│
├── inference/
│   ├── workers/
│   ├── preprocessing/
│   ├── postprocessing/
│   └── model/
│
├── stores/
│
├── hooks/
│
├── types/
│
├── tokens/
│
├── utils/
│
└── assets/
```

---

72. Core Component Inventory

The component library should explicitly define:

Application

- `AppShell`
- `TopBar`
- `Sidebar`

- `MobileNavigation`

Inspection

- `CameraViewport`
- `SegmentationOverlay`
- `DefectMarker`
- `ActiveAnomalyPanel`
- `InspectionStatus`
- `RejectionBanner`
- `EventPlayback`

Metrics

- `MetricValue`
- `MetricStrip`
- `FTRIndicator`
- `PPMIndicator`

System

- `CameraStatus`
- `AIStatus`
- `ConnectionStatus`
- `LineStatus`

Actions

- `ManualOverrideButton`
- `RecalibrateButton`
- `ExportReportButton`

Data

- `DefectTable`
- `BatchTable`
- `InspectionEventDrawer`

---

73. UX Priority Matrix

Very High Priority

- camera feed
- defect segmentation
- critical alert
- line status

- rejection status
- active defect information
- camera health

High Priority

- FTR
- PPM
- inspected count
- defect history
- event playback
- manual override

Medium Priority

- analytics
- reports
- calibration
- system diagnostics

Lower Priority

- advanced configuration
- historical comparisons
- administrative settings

The live operator experience must never be overloaded with lower-priority information.

---

74. Frontend Performance Targets

The frontend should preserve the SRS performance budget.

Targets:

```
UI interaction latency: <100 ms where practical

Inference:
≤35 ms

Total inspection pipeline:
<80 ms

Rendering:
60 FPS target for UI overlay
```

```
Browser heap:
<450 MB during continuous operation
```

The SRS explicitly defines the inference and total latency budgets and a browser heap target.

---

75. Performance Rules

Never:

- render every inference result as a React state update
- recreate large arrays every frame
- store video frames in global state
- trigger unnecessary component trees from camera updates
- perform expensive image processing on the main thread

Prefer:

- Web Workers
- Canvas/WebGL
- typed arrays
- memoization
- throttled UI updates
- object reuse
- ring buffers

---

76. Security UI

Sensitive actions should require authentication.

Examples:

```
Change calibration
Change rejection settings
Change model
Disable rejection
Change thresholds
```

The SRS specifies supervisor authentication for critical calibration/rejection configuration.

---

77. Audit Trail UI

For auditable actions display:

```
Action
Changed calibration

User
QA_SUPERVISOR

Timestamp
2026-09-27 22:18:42

Previous
12.38 px/mm

New
12.42 px/mm
```

The frontend should make auditability visible without exposing cryptographic implementation details to ordinary operators.

---

78. Microcopy Rules

Use concise industrial language.

Prefer:

```
CAMERA DISCONNECTED
```

over:

```
Oops! Something went wrong with your camera.
```

Prefer:

```
REJECTION ACTUATED
```

over:

```
We've successfully processed this item.
```

Prefer:

```
LINE INTERRUPT WARNING
```

over:

```
Please be aware that something may be wrong.
```

---

79. Status Vocabulary

Use the same vocabulary everywhere.

```
RUNNING
WARNING
DEGRADED
CRITICAL
REJECTED
ACTUATED
STOPPED
DISCONNECTED
RECONNECTING
READY
INITIALIZING
```

Do not alternate between:

```
Offline
Disconnected
Not Connected
No Connection
```

for the same state.

---

80. Final Frontend Experience

The finished FabriScan frontend should feel like a **modern industrial vision-control system**, not a conventional web dashboard.

The visual hierarchy should be:

```
1. WHAT IS HAPPENING NOW?
          ↓
2. IS THE LINE SAFE / HEALTHY?
          ↓
3. WHAT DID THE AI DETECT?
          ↓
4. WHAT ACTION DID THE SYSTEM TAKE?
          ↓
5. DOES THE OPERATOR NEED TO ACT?
          ↓
6. WHAT DOES THE PRODUCTION DATA SAY?
          ↓
7. WHAT HAPPENED HISTORICALLY?
```

The most important screen should therefore remain the **Live Inspection screen**.

Everything else exists to support:

- operation
- investigation
- quality management
- traceability
- maintenance
- reporting

rather than competing with the live inspection experience.

---

81. Design Definition of Done

The frontend should not be considered complete merely because all pages exist.

It is complete when:

- the operator understands line state immediately
- camera status is unmistakable
- defects are visually localized

- severity is immediately understandable
- rejection state cannot be missed
- critical actions require deliberate interaction
- mobile camera inspection works
- desktop/panel-PC inspection works
- UI remains usable during degraded connectivity
- camera loss produces the required warning state
- low FPS produces the required warning state
- live rendering remains performant
- WebSocket status is visible where relevant
- permissions are respected
- audit-sensitive actions are traceable
- all controls have consistent icons
- all icon-only controls have accessible labels/tooltips
- the application works without Internet access
- visual hierarchy remains clear at different resolutions

---

82. Design Philosophy

FabriScan's frontend should ultimately communicate one idea:

> **The machine sees the defect. The operator immediately understands what happened. The system takes the appropriate action. Every event remains traceable.**

The UI should therefore prioritize **clarity over decoration, signal over noise, and operational confidence over visual novelty**.

This is the frontend foundation on which the actual FabriScan prototype should be designed.